RQ1 v1.3 — Evidence-Grounded Semantic Recovery for Stripped Type-II/III Firmware

Revised planning document (vertical-slice-first) with raw .bin bootstrapping details, worked examples, stage-level validation, and a cross-format case study (ELF vs raw .bin).

Date: March 1, 2026
Project: BinAgent (Binary Analysis)
Document owner: [Your Name]

Version history

Version	Date	Author	Notes
v1.0	2026-02-23	[Your Name]	Initial detailed plan (waterfall-style milestones, full taxonomy).
v1.1	2026-02-27	[Your Name]	Revision after feedback: vertical-slice-first roadmap; explicit tool boundaries (Ghidra vs custom); difficulty-staged sinks; LLM reproducibility & cost controls; BinAgent integration; dataset/ground-truth protocol; go/no-go criteria; per-stage examples & validation.
v1.2	2026-02-27	[Your Name]	Added explicit support for stripped raw .bin firmware (vector-table bootstrap, base/memory-map inference, and decompiler boundaries). Expanded DMA handling with a raw-binary worked example and documented failure modes + validation protocol.
v1.3	2026-03-01	[Your Name]	Added an explicit cross-format case study (unstripped ELF vs stripped ELF vs raw .bin) for the same vulnerable firmware. Includes a worked source-identification example, concrete build artifacts, disassembly/representation deltas, and a tooling/custom/LLM

capability matrix.

1. Executive summary

RQ1 asks: in stripped Type-II/III monolithic firmware (missing symbols/types, weak structure metadata), how can we automatically recover and semantically label (i) hardware-dependent sources (MMIO reads, interrupt-driven buffers, DMA-backed buffers) and (ii) dangerous sinks (unsafe copies and memory writes), while keeping the automation trustworthy?

This plan operationalizes the core design principle: LLM proposes, static analysis verifies. The system is fail-closed: any label not supported by evidence and label-specific verification obligations is downgraded to PARTIAL or UNKNOWN rather than guessed.

Compared to the original plan (v1.0), this revision prioritizes an early end-to-end vertical slice (starting with MMIO_READ) so that we can validate architecture, evidence packaging, and verifier obligations before scaling to more label types.

1.1 Intended technical contributions

The publishable contributions are expected to be:

- A firmware-specific, evidence-grounded semantic labeling layer for sources and sinks in stripped monolithic firmware.
- A constrained summary language (JSON schema) that is compositional and verifier-friendly.
- A label-specific obligation system (static checks) that filters hallucinations and reduces false positives.
- A stage-wise evaluation protocol and benchmark construction methodology for Type-II/III settings.

1.2 Scope and non-goals

- In-scope: static recovery and labeling of sources/sinks; auditable evidence; fail-closed verification; integration into BinAgent as a reusable label/fact layer.
- In-scope: limited on-demand deep checks (bounded symbolic execution / value-set) only as a verifier fallback, not as a core requirement.
- Non-goal: full device rehosting or peripheral emulation as a prerequisite (can be used for optional confirmation only).
- Non-goal: solving all alias analysis precisely; we adopt scalable approximations with explicit confidence and failure modes.

2. Output contract: labels, schemas, and verdicts

The system outputs two primary artifacts: (1) source labels and (2) sink labels. Each label is a structured record with evidence references, a verification verdict, and an auditable trail.

2.1 Label taxonomy (v1.2)

We keep a stable taxonomy, but implement it in difficulty-staged order (Section 4). Label names are intentionally coarse: the goal is reliable semantic categorization, not peripheral identification.

Category	Label	Meaning	Difficulty / stage
Source	MMIO_READ	Value originates from a load from a constant MMIO base/range.	Stage VS0 (first)
Source	ISR_MMIO_READ	MMIO_READ occurring in ISR context (stronger evidence of external input).	Stage VS2
Source	ISR_FILLED_BUFFER	A global/static buffer is written in ISR and later read/parsed outside ISR.	Stage VS2
Source	DMA_BACKED_BUFFER	A memory object is the destination of a DMA transfer configured by firmware.	Stage VS3 (Level 1-3)
Sink	COPY_SINK	Dangerous copy callsite (e.g., memcpy/strcpy) or recognized copy idiom.	Stage VS1 (API/sig), later inline
Sink	MEMSET_SINK	Memory set callsite with variable length/target.	Stage VS4
Sink	STORE_SINK	A store through pointer or base+offset that may be attacker-influenced.	Stage VS4
Sink	LOOP_WRITE_SINK	Loop-based write to memory with computed bounds/index (hard).	Stage VS5 (last)

2.2 Constrained summary language (JSON schema)

Every LLM output must conform to a constrained JSON schema. All key claims must cite evidence IDs (E1..En) from the evidence pack. If parsing fails, the proposal is rejected (fail-closed).

```
// SourceLabel (simplified)
{
  "label": "MMIO_READ | ISR_MMIO_READ | ISR_FILLED_BUFFER |
DMA_BACKED_BUFFER | UNKNOWN",
  "site": {"func": "...", "addr": "0x...", "bb": "0x..."},
  "object": {"base": "0x...", "range": ["0x..","0x.."], "notes":
"..."} ,   // for buffer labels
  "io_addr": "0x..." ,
// for MMIO labels
  "claims": [ {"type": "CONST_ADDR_LOAD", "evidence_refs":
["E1","E3"]}, ... ],
  "confidence": 0.0,
  "evidence_refs": ["E1","E2"],
  "notes": "optional, short"
}
// SinkLabel (simplified)
{
  "label": "COPY_SINK | MEMSET_SINK | STORE_SINK | LOOP_WRITE_SINK |
UNKNOWN",
  "site": {"func": "...", "addr": "0x...", "bb": "0x..."},
  "dst_expr": "...",
  "len_expr": "...",
  "guard_summary": {"has_bounds_check": true, "evidence_refs":
["E4"]},
  "confidence": 0.0,
  "evidence_refs": ["E2","E4"],
  "notes": "optional, short"
}
```

2.3 Verification verdicts and how they count in metrics

Each proposed label is assigned one of three verdicts. To avoid metric inflation, PARTIAL labels are reported separately and do not count as VERIFIED true positives by default.

Verdict	Meaning	Default scoring
VERIFIED	All required obligations pass; evidence supports the claim.	Counts as TP/FP in main tables.
PARTIAL	Core category plausible but some required fields/claims unverifiable.	Reported separately; not counted as VERIFIED TP.
REJECTED/UNKNOWN	Evidence inconsistent or insufficient; or LLM output invalid.	Excluded from positives; analyzed as failures.

3. System architecture (with explicit tool boundaries)

3.1 High-level pipeline

The pipeline is designed to be modular and auditable. Candidate mining is deterministic; LLM is only used for semantic labeling; verification is static and fail-closed.

```
Firmware
  -> Loader/Normalizer (architecture, base address hypotheses)
  -> Lifting/IR (Ghidra headless: disasm, CFG, decompile/p-code,
xrefs)
  -> Program facts (custom: MemoryAccessIndex + bounded slicing over
p-code)
  -> Candidate miners (custom: sources + sinks)
  -> Evidence packer (custom: stable evidence IDs, compression)
  -> LLM proposer (optional, budgeted; constrained JSON)
  -> Verifier (custom obligations + optional deep checks)
  -> Label store + BinAgent integration (facts DB + APIs)
```

3.2 Boundary: what we use from Ghidra vs what we build

To avoid re-implementing mature components, we adopt Ghidra headless as the default front-end.

Capability	Provided by Ghidra	Custom layer (this project)
Loading / basic disassembly	Yes	Wrapper scripts; base-address hypotheses & scoring
Functions, basic blocks, CFG	Yes (best-effort)	Confidence tracking; post-filters for noisy functions
Cross-references (strings, data)	Yes	Normalization; provenance labels for xref kinds
Decompilation / p-code	Yes	Evidence extraction formatting; stable IDs
SSA / def-use	Partial / not turnkey	Bounded slicing engine over p-code; simplified SSA where needed
Memory access indexing	No	MemoryAccessIndex with base provenance & range clustering
Label verification obligations	No	Obligation checks (dominance/def-use/addr-range consistency)

3.2.1 What the decompiler gives us on raw .bin (and what it does not)

Ghidra's decompiler and analysis can still be very effective on stripped raw binaries, but only after we supply the missing program context (architecture, base address, and memory layout). In other words: for .bin, the decompiler is not the first step — bootstrapping is.

What we rely on Ghidra to provide (once bootstrapped):

- Disassembly + instruction semantics for the configured ISA (e.g., ARM Thumb/Thumb-2).
- Function discovery, CFG/basic blocks, and an approximate call graph (not perfect, but usable).
- Decompilation to p-code and C-like pseudo-code (used as the analysis substrate for MemoryAccessIndex and slicing).
- Cross-references (xrefs) for constants, strings, and data/code references.
- P-code-level def-use within a function (enough to build bounded slices).

What Ghidra cannot infer from a raw .bin by itself (and what we must pre-deal):

- Load address / flash base (no ELF headers), and therefore absolute addresses for code and literals.
- Section boundaries (.text/.data/.bss) and which ranges should be treated as executable vs data.
- RAM/peripheral memory maps (needed to interpret 0x2000… as SRAM and 0x4000… as MMIO).
- Where the vector table begins/ends and which entries are real ISR handlers vs accidentally reading into code.
- MCU-family-specific register maps (critical for DMA and for distinguishing control vs data registers).

In this project, the Loader/Normalizer (Stage 1) is responsible for turning an unstructured .bin into a well-formed "program" for Ghidra: choosing a base-address hypothesis, creating memory blocks (flash/RAM/peripheral), seeding entry points (reset handler + ISRs), and recording the hypothesis so downstream obligations can reason about it.

3.2.2 Raw-binary pre-deal checklist (before Ghidra auto-analysis)

- Parse a candidate vector table (if applicable) and derive initial SP, reset handler, and active ISR entries.
- Generate multiple base-address / memory-map hypotheses (e.g., 0x08000000 vs 0x00400000) and score them by internal consistency (valid code pointers, CFG quality, reasonable ISR set).
- Create Ghidra memory blocks: flash (backed by bytes), RAM/peripheral (uninitialized blocks) and mark permissions (RX vs RW).
- Seed function starts at reset handler and ISR handlers; optionally add common prologue patterns as additional seeds.
- Run Ghidra auto-analysis deterministically (fixed options), store metrics and chosen hypothesis in the artifact store.

```
// Example (Cortex-M raw blob bootstrapping)
// word[0] = 0x20050000  -> initial SP (SRAM range)
// word[1] = 0x080011ED  -> reset handler (odd => Thumb) =>
```

```
flash_base candidate 0x08000000
// word[2..N]            -> ISR handlers (odd addresses within flash
range)
// Use "odd + in-range" validity to infer vector table length and
reject reading past the table.
```

3.3 Data stores and integration surfaces

All intermediate artifacts are persisted to support debugging, reproducibility, and integration into BinAgent.

Store	Contents	Used by
facts.sqlite	Functions/CFG, xrefs, MemoryAccessIndex, slices	miners, evidence packer, verifier
candidates.sqlite	All source/sink candidates + features + ranks	LLM budgeter, evaluation
proposals.jsonl	Raw LLM outputs + parsed JSON + config metadata	verifier, reproducibility
labels.jsonl	Final verdicted labels (VERIFIED/PARTIAL/UNKNOWN)	BinAgent + experiments

4. Roadmap: vertical-slice-first (revised milestones)

Instead of finishing all infrastructure milestones before seeing results, we iterate vertically: implement one label type end-to-end (mine -> pack -> propose -> verify -> evaluate), then expand to additional labels. Each vertical slice has explicit go/no-go criteria.

4.1 Vertical slices (VS0–VS5)

Slice	New capability	Key obligations (examples)	Stage demo	Go/No-Go metrics
VS0	MMIO_READ end-to-end (miner + evidence + verifier)	O_MMIO_1 const base; O_MMIO_2 non-code/data	Given a UART driver sample, label the data-register read as MMIO_READ with evidence.	MMIO precision >= 0.8 on microbench; LLM parse fail < 5%
VS1	Add COPY_SINK (API/sig) + connect to MMIO sources	O_COPY_1 known API/sig; O_COPY_2 args recoverable	Find memcpy/strcpy sink in sample and show source->sink candidate path exists.	COPY precision >= 0.85; verified-rate >= 0.6 on controlled set
VS2	ISR context + ISR_FILLED_BUFFER	O_ISR_1 ISR entry evidence; O_BUF_1 ISR writes; O_BUF_2 non-ISR reads	Identify ISR writes into a ring-like buffer and a task reads it.	ISR detection recall >= 0.8 on Cortex-M samples; buffer FP manageable (<= 2x TP)
VS3	DMA_BACKED_BUFFER Level 1–2	O_DMA_1 config writes; O_DMA_2 dst pointer traces to object	Detect DMA config and mark dst buffer as DMA_BACKED_BUFFER.	DMA Level-1 recall >= 0.6 on selected DMA examples; verified labels >= 0.4
VS4	Add MEMSET_SINK + STORE_SINK	O_MEMSET_1 callsite; O_STORE_1 pointer store expr	Report verified sinks beyond copy calls (e.g., memset with variable len).	Maintain precision >= previous slices; incremental recall improves
VS5	LOOP_WRITE_SINK (hard mode)	O_LOOP_1 loop structure; O_LOOP_2 index/len recovered	Catch an inlined loop copy/write that is not a library call.	At least N verified examples on microbench; document failure modes

4.2 Minimal waterfall milestones (for tracking only)

Internally we still track engineering milestones (loader, facts, miners, evidence, proposer, verifier, evaluation), but implementation order follows the vertical-slice plan above.

Milestone	Deliverable	Relative effort	Validation hook
M0	Evaluation harness + microbench generator	Medium	Run unit tests; compute precision/recall on microbench
M1	Loader + memory-map hypotheses	Medium	Correctly locate segments and avoid MMIO/flash confusion
M2	Program facts: MemoryAccessIndex + slicing	High	Golden slicing tests on synthetic binaries
M3	Sink miners (staged)	Medium-High	COPY_SINK unit tests first; LOOP_WRITE later
M4	Source miners (staged)	Medium-High	MMIO first; ISR then DMA
M5	Evidence packer with stable IDs	Medium	Deterministic pack for same input (hash-stable)
M6	LLM proposer (budgeted + reproducible)	Medium	Re-run yields identical parsed JSON outputs
M7	Verifier obligations + deep-check fallback	High	Ablation: precision improves with obligations
M8	Storage/reporting + BinAgent integration	Medium	BinAgent can query labels; callsite queue populated
M9	End-to-end evaluation + ablations + case studies	High	Paper-ready results tables/figures

5. Experimental design, datasets, and ground truth

5.1 Dataset strategy: three tiers

Because Type-II/III firmware often lacks ground truth and structure metadata, we use a tiered dataset plan so that every stage can be validated early, and the final system can still be evaluated on realistic blobs.

Tier	Purpose	Examples (suggested)	Ground truth method
T0: Microbench	Unit tests for each label + obligation; deterministic patterns	Tiny bare-metal C programs with explicit MMIO constants, ISR stubs, memcpy calls, inlined loops, DMA descriptor mockups	We author the program; ground truth is exact by construction; compile both debug + stripped.
T1: Source-available firmware samples	Stage demos and quantitative evaluation on realistic codebases	Zephyr samples (drivers/uart, gpio), FreeRTOS/Amazon FreeRTOS demos, STM32 HAL examples, small RTOS apps	Compile from source twice (debug and stripped). Use debug symbols/map files to extract oracle labels.
T2: Real-world firmware blobs	Stress-test robustness under missing metadata and vendor idiosyncrasies	Fuzzware/P2IM targets; vendor firmware images when available	No full oracle. Use partial validation: manual spot-check + cross-tool consistency + optional dynamic confirmation.

5.2 Ground truth extraction protocol (T0/T1)

For T0/T1, we generate an oracle from the source/debug build, then evaluate on a stripped build of the same program.

- Build A (oracle build): keep debug symbols, generate map file; disable aggressive LTO when it destroys traceability.
- Build B (target build): strip symbols; export both stripped ELF (when applicable) and raw .bin; optionally vary compiler/optimization to test generalization.
- Oracle extraction: collect (a) MMIO constants used in volatile loads, (b) ISR vector table entries, (c) DMA destination buffers where visible, (d) library callsites for memcpy/memset, and (e) known inlined copy loops.
- Address matching: use map/ELF symbol addresses to map oracle labels to stripped addresses (same build settings).
- Annotation log: store scripts + commit IDs so the oracle is reproducible.

5.3 Evaluation baselines and ablations

Baselines (reported on T0/T1; partial on T2):

- Heuristic-only: miner labels without LLM; verifier still applied where possible.
- LLM-only: LLM labels accepted without obligations (upper-bound on hallucination risk).
- Proposed: LLM + verifier obligations (fail-closed).

Ablations (paper-ready):

- Remove slicing/context from evidence packs (does grounding matter?).
- Remove obligations (accept all LLM proposals).
- Reduce obligation set (which obligations matter most?).
- No-LLM fallback mode (miner + verifier only).

5.4 Reproducibility requirements (especially for LLM)

- Record model name and version, temperature/top_p, max tokens, retry policy, and any system prompts.
- Deterministic post-processing: strict JSON parsing, canonical key ordering, stable sorting for candidates.
- Cache by evidence-pack hash (avoid re-querying the same candidate).
- Budget rules are part of the experiment config (Top-K per label type, max tokens per firmware).

6. Stage-by-stage implementation plan (with examples and validation)

This section is the execution playbook. Each stage includes: (a) what to build, (b) the concrete approach, (c) a worked example, and (d) how to validate correctness at that stage.

6.0 Stage 0 — Microbench suite + evaluation harness (M0)

Goal: create a controlled testbed where each label type and obligation has deterministic ground truth.

6.0.1 Approach

- Write a small set of bare-metal C programs, each designed to contain exactly one pattern (e.g., MMIO read, memcpy sink, ISR buffer write).
- Compile each program into (A) an oracle build with symbols and (B) a stripped build for analysis.
- Export oracle labels automatically from the oracle build (e.g., using nm/objdump, map files, or compile-time logging macros).
- Implement eval_runner that compares predicted labels (VERIFIED only, by default) against oracle labels at site-level and object-level.

6.0.2 Example microbench (MMIO + COPY)

```
// mmio_copy.c (T0 example)
#define UART_DR   (*(volatile unsigned int*)0x40011004u)

void handler(char* dst, unsigned int n) {
  unsigned int x = UART_DR;        // MMIO_READ source
  if (n < 16) n = 16;
  memcpy(dst, &x, n);              // COPY_SINK sink (potential
overflow if dst small)
}
```

Expected oracle labels:

- - Source: MMIO_READ at the load from 0x40011004
- - Sink: COPY_SINK at the memcpy callsite

6.0.3 Stage validation (how we know this stage is correct)

- Unit test: eval_runner should report 100% precision/recall when fed the oracle labels directly.
- Golden test: run the pipeline on mmio_copy stripped binary; verify that at least the MMIO_READ candidate is mined and the evidence pack includes the constant address load.
- Sanity metric: LLM parse success on T0 should be ~100% (schema is simple).

6.1 Stage 1 — Loader/Normalizer + memory-map hypotheses (M1)

Goal: robustly load monolithic firmware blobs and infer plausible base addresses/segment partitions so that address-based reasoning (MMIO vs flash vs SRAM) is meaningful.

6.1.1 Approach

- Support input formats: raw blob and ELF (when available). For raw blobs, require architecture/endianness config or implement a best-effort detector.
- Generate a small set of memory-map hypotheses: e.g., flash_base candidates, RAM ranges, and peripheral ranges (MCU-specific).
- Score hypotheses using internal consistency signals (e.g., CFG quality, number of valid code references, plausible vector table location).
- Persist the chosen hypothesis and top alternatives; downstream obligations can consult them.

6.1.2 Example: base-address hypothesis scoring

```
// Example scenario (Cortex-M raw blob)
Hypothesis A: flash_base = 0x08000000 (common STM32)
Hypothesis B: flash_base = 0x00000000 (flat mapping)

Score features:
- Vector table looks plausible? (first word = initial SP in SRAM
range; second word = reset handler in flash range)
- How many instructions decode cleanly near reset handler?
- How many xrefs land in-range vs out-of-range?

Pick hypothesis with highest score; keep both for debugging.
```

6.1.3 Stage validation

- On T1 Cortex-M samples with known flash base, hypothesis scorer should pick the correct base >90% of the time.
- Regression test: re-running loader with same inputs yields identical hypothesis choice (deterministic).
- Manual spot-check: vector table parsing matches expected ISR entries for at least 3 samples.

6.1.4 Raw .bin bootstrapping: from bytes to decompilation (worked example)

Type-II/III firmware is often distributed as a stripped raw blob (.bin) with no ELF headers, no section table, and no load address. The plan above supports “raw blob” inputs; this subsection makes the .bin workflow explicit and testable.

What changes for raw .bin (vs ELF): we lose metadata, but we do not lose the machine-code patterns that drive source/sink recovery. In particular, vector tables, literal pools, instruction idioms, and string constants survive stripping.

Bootstrapping pipeline (Cortex-M first; other ISAs fall back to configuration or multi-ISA scoring):

- Step 1 — Architecture + endianness inference: if a Cortex-M vector table is present, infer ISA from (SP in 0x2000… SRAM range) and (reset handler address has Thumb bit set: odd). Otherwise, require explicit config or run competing ISA loaders and pick the one with the highest internal-consistency score.

- Step 2 — Base-address hypotheses: derive candidate flash bases by aligning the reset handler down to common flash bases (e.g., 0x08000000, 0x00400000) and/or by scanning for self-consistent code pointers in the first N words.
- Step 3 — Vector table length inference: treat each 32-bit entry as "valid" if it is 0 or (odd + within the hypothesized code range). Stop when the validity test fails to avoid reading past the table into code (a common failure mode in raw .bin).
- Step 4 — Memory blocks for Ghidra: create (a) flash at chosen base (backed by bytes, RX), (b) RAM (RW, uninitialized), and (c) peripheral/MMIO ranges (RW, uninitialized). This enables pointer/range reasoning even without real bytes for RAM/MMIO.
- Step 5 — Deterministic auto-analysis + scoring: run Ghidra auto-analysis with fixed settings; score the hypothesis using CFG quality, number of reachable basic blocks, number of valid code xrefs, and plausibility of ISR entry points.

Worked example (illustrative): vector table → base address → analysis setup

```
// Suppose the first 8 words of a raw .bin are:
//   w0 = 0x20050000  (initial SP => SRAM)
//   w1 = 0x080011ED  (reset handler; odd => Thumb)
//   w2 = 0x08001A31  (ISR handler)
//   w3 = 0x00000000  (unused)
//
// Base candidates:
//   flash_base_A = 0x08000000  (common STM32)
//   flash_base_B = 0x00400000  (some QEMU/SoCs)
//
// For each base candidate:
//   - infer code range [flash_base, flash_base + file_size)
//   - count how many vector entries are "valid" (0 or odd+in-range)
//   - run Ghidra analysis and score CFG reachability
// Choose the hypothesis with the best combined score and persist it.
```

Practical note: naive scanning for "any 32-bit word in the peripheral range" is extremely noisy on raw .bin (data values frequently fall in 0x4000… by coincidence). A key filter is to only consider constants that are actually used as addresses in code (e.g., PC-relative LDR literal loads) and that feed into LOAD/STORE p-code ops. This reduces false positives by orders of magnitude.

```
// Example filter idea
// Naive: count all words v where 0x40000000 <= v < 0x60000000  =>
thousands of hits
// Better: count only v that appear in literal pools referenced by
LDR (PC-relative) and are used as a pointer base in p-code LOAD/STORE
=> tens of hits
// Then cluster by (base & ~0xFF) to find peripheral groups.
```

Validation checklist (how we know the raw .bin bootstrap is correct):

- Paired-build check: for each T0/T1 program, compile both an ELF and a raw .bin variant from the same source and compare recovered MMIO_READ sites; the .bin result should match the ELF result up to expected function-boundary noise.

- Vector-table sanity: recovered ISR entry points must all be (odd + in code range) and should produce non-empty, short handler functions after decompilation.
- Determinism: re-running the loader on the same .bin yields the same chosen hypothesis and identical label IDs (or stable hashes).

6.2 Stage 2 — Program facts: MemoryAccessIndex + bounded slicing over p-code (M2)

Goal: extract the minimal static facts needed to (a) mine candidates and (b) verify LLM claims: memory accesses with base provenance, def-use chains, and bounded slices.

6.2.1 Approach

- Build MemoryAccessIndex: enumerate all loads/stores in p-code with (addr_expr, width, read/write, function, block).
- Track base provenance: classify addr_expr base as CONST, GLOBAL_PTR, STACK_PTR, ARG, RETURN, or UNKNOWN.
- Implement bounded backward/forward slice: use p-code def-use to trace up to K steps or until reaching a root (CONST/ARG/GLOBAL).
- Expose APIs: slice(site) -> {defs, uses, guard predicates, loop context}.

6.2.2 Example: recording a memory access with provenance

```
// Assembly-like example
ldr r0, =0x40011000
ldr r1, [r0,#0x04]      // load from base CONST + offset

MemoryAccessIndex entry:
- kind: LOAD
- addr_expr: CONST(0x40011000) + 0x04
- base_provenance: CONST
- width: 32
- site: (func=uart_read, addr=0x..., bb=0x...)
- context: in_loop? yes/no; guards: [cmp r1,#...]
```

6.2.3 Stage validation

- Golden slicing tests on T0: given a known MMIO load, backward slice must reach CONST base within K steps.
- Negative test: constant-table read from flash should be classified as CONST base but later filtered by addr-range obligations (Stage 6.7).
- Performance check: indexing and slicing scale roughly linearly in function size for T1 samples.

6.3 Stage 3 — Source mining: MMIO_READ vertical slice (VS0)

Goal: reliably mine MMIO_READ candidates and produce VERIFIED labels using minimal LLM assistance.

6.3.1 Approach (engineering baseline)

- Candidate generation: from MemoryAccessIndex, select loads whose addr_expr has CONST base or traces to CONST within K-step backward slice.
- Clustering: group addresses into 'MMIO clusters' by base range and proximity (helps reduce duplicates and supports peripheral-agnostic reasoning).
- Context features (for ranking): presence of bitmask ops (&, >>), polling loops (repeated reads), or read-then-write-to-same-cluster patterns.
- LLM usage (optional in VS0): only to classify 'MMIO-like vs flash-table-like' with evidence citations; verifier is final arbiter.

6.3.2 Worked example (MMIO read vs flash table)

```
// Candidate A (MMIO-like)
ldr r0, =0x40011000
ldr r1, [r0,#0x04]        // read UART status/data reg
ands r1, r1, #0x80
beq  loop

// Candidate B (flash constant table)
ldr r2, =0x08001000
ldr r3, [r2,#0x10]        // read constant table in flash
add r4, r3, r5
```

How the system distinguishes them:

- Both have CONST base provenance, so both become candidates.
- Verifier uses memory-map hypothesis: 0x4001xxxx is in peripheral/MMIO range; 0x0800xxxx is flash/code/data range.
- Additional context: Candidate A is in a polling loop and uses bit tests; Candidate B is used as arithmetic constant.

6.3.3 Stage validation (quantitative + demo)

- T0: precision/recall on synthetic MMIO programs should be high (target >=0.8 precision).
- T1: pick 3-5 driver-like samples (e.g., UART polling) where MMIO constants are visible in source headers. Use the oracle build to extract the MMIO address constants and compare.
- Demo correctness check: for each VERIFIED MMIO_READ, the evidence pack must include (E1) the load instruction, (E2) the backward slice reaching CONST base, and (E3) addr-range check result.

6.4 Stage 4 — Sink mining (staged): COPY_SINK first (VS1)

Goal: identify dangerous copy sinks with a staged approach to avoid early overreach. Start with explicit library calls or import-table signatures; add inlined copies later.

6.4.1 Approach (Stage S1: API/sig copy calls)

- If symbols/imports exist: directly match memcpy/memmove/strcpy/strncpy/sprintf-like patterns.
- If stripped: apply function-signature matching on small candidate functions or common PLT stubs; fallback to idiom-based detection (argument movement patterns).
- Extract arguments (dst/src/len) using calling convention + local def-use slice.
- Rank sinks: variable length, pointer destination, and missing obvious guard increase priority.

6.4.2 Worked example (COPY_SINK detection)

```
// Decompiled-style snippet
void f(char* dst, char* src, int n) {
  if (n > 256) n = 256;
  memcpy(dst, src, n);     // COPY_SINK
}
```

Evidence captured:

- E1: callsite to memcpy (or recognized signature stub)
- E2: slice for argument n (shows clamp or lack thereof)
- E3: slice for dst pointer (stack/global/arg provenance)

6.4.3 Stage validation

- T0: unit tests for known memcpy callsites (with -fno-builtin to preserve calls).
- T1: compile samples with and without libc builtins; measure detection robustness.
- Metric: COPY_SINK precision >= 0.85 on T0/T1; report split for (symbolic vs stripped signature).

6.5 Stage 5 — Evidence packer: stable, compressed, and auditable (M5)

Goal: convert raw program facts into a compact evidence pack that is (a) human-auditable, (b) LLM-readable, and (c) stable across runs (same candidate -> same evidence IDs).

6.5.1 Approach

- For each candidate site, include: local code window, backward slice (K hops), forward slice (K hops), and CFG context (loop/guards).
- Assign stable evidence IDs by hashing normalized p-code statements + their roles (e.g., 'E1: site instruction', 'E2: base definition').
- Include a small structured 'facts' section (addr_expr, base provenance, segment classification, in_ISR flag) to avoid long prose.
- Apply compression rules when slices are large: keep only the backbone of def-use chain and the closest control predicates.

6.5.2 Worked example: evidence pack for a MMIO_READ candidate

```
// EvidencePack (excerpt)
{
  "candidate_id": "mmio@0x100084",
  "candidate_hint": "MMIO_READ",
  "facts": {
    "addr_expr": "CONST(0x40011000) + 0x04",
    "segment_of_base": "PERIPHERAL_RANGE",
    "in_loop": true,
    "in_isr": false
  },
  "evidence": [
    {"id":"E1","type":"SITE","text":"ldr r1, [r0,#0x04]  // read"},
    {"id":"E2","type":"DEF","text":"ldr r0, =0x40011000"},
    {"id":"E3","type":"GUARD","text":"ands r1, r1, #0x80; beq loop"},
    {"id":"E4","type":"SLICE_BACK","text":"r0 := 0x40011000; addr :=
r0+4"}
  ]
}
```

6.5.3 Stage validation

- Determinism test: evidence pack for the same candidate should be byte-identical (or hash-identical) across two runs.
- Audit test: a human can inspect E1–E4 and decide if the label is plausible (no missing critical context).
- LLM load test: ensure the pack stays under a fixed token budget by truncation rules (record truncation in metadata).

6.6 Stage 6 — LLM proposer (budgeted, reproducible, and verifier-aware) (M6)

Goal: use LLMs only where they add value (semantic interpretation), under strict budgets and schemas. Verifier is the final arbiter; the proposer must never override failed obligations.

6.6.1 Approach

- Prompt templates are label-specific and short: the model receives the evidence pack and must output schema-valid JSON with evidence_refs.
- Budgeting: only query LLM for the top-K ranked candidates per label type; cache by evidence-pack hash.
- Batching: optionally pack 10–20 small candidates in one call when evidence packs are small and independent.
- Conflict handling: miner provides a hint; LLM may disagree; verifier decides. If verifier cannot validate either label, final is UNKNOWN.
- Reproducibility: temperature=0, deterministic decoding settings, fixed prompt text, and strict JSON parser with canonical formatting.

6.6.2 Worked example: proposer prompt and expected output

```
// Prompt (excerpt)
You are labeling a firmware candidate. Use ONLY the evidence items
provided.
Output MUST be valid JSON matching the schema. For every claim, cite
evidence_refs.

EvidencePack:
[E1] ldr r1, [r0,#0x04]
[E2] ldr r0, =0x40011000
[E3] ands r1, r1, #0x80; beq loop
Facts: addr_expr=CONST(0x40011000)+0x04;
segment_of_base=PERIPHERAL_RANGE

Task: Label as MMIO_READ or UNKNOWN and provide claims/evidence_refs.
```

```
// Expected JSON (example)
{
  "label": "MMIO_READ",
  "site": {"func":"uart_poll","addr":"0x100084","bb":"0x100080"},
  "io_addr": "0x40011004",
  "claims": [
    {"type":"CONST_ADDR_LOAD","evidence_refs":["E1","E2"]},
    {"type":"POLLING_OR_BITTEST","evidence_refs":["E3"]}
  ],
  "confidence": 0.78,
  "evidence_refs": ["E1","E2","E3"],
  "notes": "Load from peripheral-range constant base; used in bit-
test loop."
}
```

6.6.3 Stage validation

- Schema compliance: JSON parse success rate and required fields present.
- Budget compliance: average tokens/candidate and max cost per firmware are recorded and enforced.

- Stability: rerunning on same evidence packs yields identical JSON (after canonical formatting).

6.7 Stage 7 — Verifier: obligation system + deep-check fallback (M7)

Goal: turn LLM proposals into trustworthy labels by enforcing label-specific obligations. Verifier is designed to be cheap by default and only triggers deeper checks for high-value borderline cases.

6.7.1 Obligation design principles

- Obligations are local, checkable, and tied to program facts (def-use, CFG, addr-range).
- Each label defines a minimal required set; optional strengthening obligations improve confidence but are not mandatory.
- Fail-closed: if any required obligation fails, verdict becomes REJECTED/UNKNOWN (or PARTIAL if category plausible but details missing).

6.7.2 Example obligations (initial set)

Label	Required obligations	Optional strengthening	Common failure modes
MMIO_READ	O_MMIO_1: addr_expr traces to CONST base via slice; O_MMIO_2: base/range classified as peripheral/MMIO (not code/data)	O_MMIO_3: value used in bit-test/polling; O_MMIO_4: cluster appears in multiple functions	Flash constant table misclassified; missing/incorrect memory-map hypothesis
COPY_SINK	O_COPY_1: callsite matches known API or verified signature; O_COPY_2: dst/src/len expressions extracted or marked UNKNOWN with low confidence	O_COPY_3: len is variable and lacks dominating bounds check	Inlined copies not captured; calling convention ambiguity
ISR_MMIO_READ	O_ISR_1: site reachable from ISR entry (vector table evidence); plus MMIO_READ obligations	O_ISR_2: ISR clears interrupt status in same cluster	ISR identification missing due to blob format or relocated vectors
DMA_BACKED_BUFFER	O_DMA_1: evidence of DMA config write sequence; O_DMA_2: dst pointer traces to an object base/range	O_DMA_3: object later read/parsed outside DMA setup code	DMA controller variants; descriptor format diversity; pointer aliasing

6.7.3 Worked example: verifier rejecting a hallucinated MMIO label

```
// LLM proposal (incorrect):
label=MMIO_READ, io_addr=0x08001010, evidence_refs=[E1,E2]

// Verifier checks:
- O_MMIO_1 passes (CONST base traces)
- O_MMIO_2 fails because 0x0800xxxx is FLASH/CODE range under current
memory-map hypothesis

Verdict: REJECTED/UNKNOWN
Reason: addr-range inconsistency (likely constant table read).
```

6.7.4 Deep-check fallback (optional)

- Trigger only for high-priority candidates (e.g., top-ranked sinks) where obligations almost pass.
- Examples: bounded symbolic execution to confirm addr_expr range; value-set analysis for pointer base recovery.
- All deep-check results are stored as additional evidence IDs and logged for reproducibility.

6.7.5 Stage validation

- Ablation: compare precision/recall with obligations enabled vs disabled (LLM-only).
- Obligation coverage: report which obligations fail most often and why (error taxonomy).
- Determinism: verifier outputs are deterministic given fixed facts DB and proposal JSON.

6.8 Stage 8 — ISR context and ISR_FILLED_BUFFER (VS2)

Goal: model interrupt-driven input propagation that is common in Type-III (bare-metal) firmware and many Type-II systems.

6.8.1 Approach

- ISR identification (Cortex-M baseline): parse vector table (first entries: initial SP and reset handler; subsequent entries: ISR handlers).
- Mark ISR context: functions reachable from ISR entry are tagged ISR-context (best-effort).
- ISR_MMIO_READ: intersect MMIO_READ sites with ISR-context reachability.
- ISR_FILLED_BUFFER: find global/static objects written in ISR-context and read/parsed in non-ISR context (cross-context shared object heuristic).

6.8.2 Worked example: ISR fills a ring-like buffer

```
// Pseudocode
volatile uint8_t rx_buf[64];
volatile uint32_t head;

void USART1_IRQHandler(void) {
  uint8_t b = *(volatile uint8_t*)0x40011004;   // MMIO_READ in ISR
  rx_buf[head & 63] = b;                         // ISR writes global
buffer
  head++;
}

int main(void) {
  while (1) {
    if (tail != head) parse(rx_buf[tail & 63]);  // non-ISR
reads/parses
  }
}
```

Expected labels:

- ISR_MMIO_READ at the MMIO load inside USART1_IRQHandler
- ISR_FILLED_BUFFER for rx_buf (object-level label with write_sites in ISR and read_sites in main loop)

6.8.3 Stage validation

- T0/T1: vector table parsing should identify known ISR entries (oracle from ELF/map file).
- Cross-context object check: in controlled samples, rx_buf should be among top shared-object candidates.
- Error analysis: report how often ISR identification fails and why (relocation, wrapper ISRs, missing vectors).

6.9 Stage 9 — DMA_BACKED_BUFFER staged support (VS3)

Goal: recover DMA-backed input objects without requiring full rehosting. We implement DMA support in levels to control risk.

6.9.1 Level 1: detect DMA configuration writes

- Mine MMIO write clusters that resemble DMA controllers (multiple consecutive register writes, descriptor pointer writes).
- Look for repeated patterns: write src/dst/len/control fields, often in a tight init routine.
- Output: DMA_CONFIG_SITE candidates (internal), used to seed Level 2.

6.9.2 Level 2: recover destination buffer object (DMA_BACKED_BUFFER)

- From a suspected DMA config site, backward slice the value written to dst register/descriptor field.
- If the slice reaches a global/static base or a stable heap object, record it as the DMA-backed object range candidate.
- Strengthen confidence by locating non-DMA reads/parsing of the same object.

6.9.3 Level 3: descriptor chain and multi-buffer patterns (optional)

- Recover simple descriptor arrays (ring descriptors) by detecting loops initializing descriptor fields.
- Track stride and bounds to infer object ranges.
- This is optional for initial publication; include as future work if it becomes too costly.

6.9.4 Worked example: DMA dst pointer traces to global buffer

```
// Simplified pattern
DMA_DST_REG = 0x40020010

void dma_init(void) {
  *(volatile uint32_t*)DMA_DST_REG = (uint32_t)&rx_dma_buf[0];  //
config write
  *(volatile uint32_t*)(DMA_DST_REG+4) = 256;                   //
len
  *(volatile uint32_t*)(DMA_DST_REG+8) = 0x1;                   //
enable
}

uint8_t rx_dma_buf[256];

void main_loop(void) {
  if (rx_dma_buf[0] == 0x7E) parse_frame(rx_dma_buf);           //
consumer
}
```

6.9.5 Why DMA is hard on raw .bin (worked example + failure modes)

DMA-backed input is harder than MMIO_READ because the “source” is not a single load from a peripheral register. Instead, the device writes external data directly into RAM via a DMA engine, and the firmware later consumes that RAM region. Recognizing this statically requires connecting (1) a DMA configuration site to (2) the destination buffer object to (3) later consumption sites — all without symbols or types.

Worked example (EasyDMA-style receive; representative of many MCUs):

```
// Pseudo-C (source-level intent)
// uarte_rx_start(buf, n): configure DMA so that incoming UART bytes
land in buf.
void uarte_rx_start(uint8_t *buf, size_t n) {
  UARTE0->RXD.PTR    = (uint32_t)buf;   // DMA destination pointer
  UARTE0->RXD.MAXCNT = (uint32_t)n;     // DMA length
  UARTE0->TASKS_STARTRX = 1;           // trigger
}

// What the raw .bin typically looks like after stripping (schematic):
LDR  r3, =0x40002000          ; peripheral base (UARTE0)
STR  r0, [r3, #0x534]         ; RXD.PTR   <- buf
STR  r1, [r3, #0x538]         ; RXD.MAXCNT<- n
MOVS r2, #1
STR  r2, [r3, #0x000]         ; TASKS_STARTRX <- 1
```

Why this is difficult on stripped raw binaries:

- MCU-family dependence: without identifying the MCU (or target memory map), 0x40002000 + 0x534 is just “some MMIO write,” not “DMA destination pointer.” A wrong peripheral map can turn true DMA into false positives (or vice versa).
- No types, no structs: decompilers often show pointer arithmetic (base+offset stores) without telling you which field is PTR vs LEN vs CTRL. The same pattern also appears in non-DMA peripheral configuration code.
- Indirection and aliasing: the value written to the PTR register may not be a direct global (&rx_buf) — it may be passed through multiple wrappers, stored in a struct, or computed as buf + offset. Bounded slices can hit UNKNOWN roots quickly.
- Descriptor chains: many DMA engines use descriptor arrays/rings. The firmware fills descriptors in a loop and then writes the head pointer to a DMA register. Without section/type info (especially in raw .bin), recovering the descriptor layout and buffer extents is substantially harder.
- Direction ambiguity: DMA is used for RX (input), TX (output), and memory-to-memory. We must avoid labeling TX buffers as input sources. This requires extra evidence (e.g., link to a peripheral RX register/task, or subsequent parsing reads from the buffer).

How our staged DMA plan handles raw .bin (and where we fail-closed):

- Level 1 (DMA_CONFIG_SITE): detect “write cluster” patterns to the same peripheral base with (pointer-like value, length-like value, enable/trigger). If an MCU register map is available, match specific offsets (PTR/LEN/CTRL). Otherwise, emit PARTIAL DMA_CONFIG_SITE with low confidence.
- Level 2 (DMA_BACKED_BUFFER): backward-slice the value written to the PTR register/descriptor field. Only accept VERIFIED when the slice reaches a stable RAM object

(GLOBAL_PTR or CONST base in RAM range). Otherwise emit PARTIAL (we found config but not a trustworthy buffer root).

- Level 2 strengthening: require at least one non-DMA consumption read of the same RAM region in main-context code (e.g., parsing loop) to reduce TX/memcpy false positives.
- Level 3 (optional): recover simple descriptor arrays by detecting loops that initialize repeated field offsets (stride inference). Treat as future work if cost is too high for the first paper.

Verifier obligations we add specifically for DMA in raw .bin:

```
// Example obligations (sketch)
// O_DMA_1: there exists a configuration cluster with >=3 writes to
the same peripheral base within a small window.
// O_DMA_2: one write value is pointer-like and slices to a RAM-range
root (GLOBAL_PTR/CONST); another is length-like (bounded int).
// O_DMA_3: there exists at least one non-DMA read from the candidate
buffer in non-ISR/main context (consumption evidence).
// If O_DMA_1 holds but O_DMA_2 fails => PARTIAL (config-only). If
O_DMA_2 holds but O_DMA_3 fails => PARTIAL (buffer-only).
```

Practical dataset/validation note: for DMA we must include paired source-available examples (T0/T1) where the DMA setup function and buffer are known, then evaluate the stripped raw .bin. This gives a reliable oracle for whether (config, dst buffer, consumption) were correctly recovered.

6.9.6 Stage validation

- T0: descriptor-free DMA examples (like above) should be recovered at Level 2 with high precision.
- T1: select a small curated set of DMA-using examples; measure Level-1 detection recall and Level-2 buffer precision.
- Document what fails: controller variants, pointer aliasing, descriptor indirection.

6.10 Stage 10 — Additional sinks: MEMSET_SINK, STORE_SINK, LOOP_WRITE_SINK (VS4–VS5)

Goal: broaden sink coverage beyond library copy calls, while keeping the staged difficulty explicit.

6.10.1 MEMSET_SINK (moderate)

- Detect memset calls (symbolic or signature) and recover (dst,val,len).
- Rank as dangerous when len is variable and the dst object size is unknown or smaller than len.

```
// Example
memset(buf, 0, n);       // MEMSET_SINK if n attacker-influenced
```

6.10.2 STORE_SINK (moderate)

- Mine stores through pointers or base+offset expressions where base provenance is ARG/GLOBAL_PTR/UNKNOWN.
- Record dst_expr and any dominating checks; do not overclaim exploitability at this stage.

```
// Example
*(p + idx) = x;          // STORE_SINK candidate when p/idx are not
constant
```

6.10.3 LOOP_WRITE_SINK (hard; last)

- Detect loops that write to memory each iteration (store in loop body).
- Try to recover loop bound and index expression via slicing; if not possible, emit PARTIAL only.
- Use microbench-heavy validation; do not expect broad coverage early.

```
// Example (inlined copy)
for (i=0; i<n; i++) dst[i] = src[i];   // LOOP_WRITE_SINK
```

6.10.4 Stage validation

- T0: dedicated microbench patterns for each sink kind; ensure obligations reject obvious non-sinks (e.g., stores to fixed globals).
- T1: measure incremental recall as sink coverage grows; keep precision stable via verifier.
- Report difficulty split: separate metrics for (API calls vs inlined loops).

6.11 Stage 11 — Storage/reporting + BinAgent integration (M8)

Goal: make RQ1 outputs usable by the larger BinAgent system. RQ1 should populate the agent's fact layer so that downstream analyses have a non-empty, high-quality starting point.

6.11.1 Integration surfaces

- Export verified labels as a facts bundle (labels.jsonl + indices) that BinAgent can load during preflight.
- Provide query APIs: get_sources(func), get_sinks(func), get_labels_by_addr(addr), get_evidence(label_id).

- Use verified sinks to populate BinAgent's callsite queue; use sources to seed taint roots and to prioritize slices.

6.11.2 Worked example: how a verified sink becomes an agent task

```
// Example: BinAgent task queue entry
{
  "task": "analyze_sink",
  "sink_label_id": "COPY_SINK@0x1012a0",
  "context": {
    "func": "handler",
    "evidence_refs": ["E1","E2","E3"],
    "source_hints": ["MMIO_READ@0x100084"]
  }
}
```

6.11.3 Stage validation

- Smoke test: BinAgent loads labels bundle and the callsite queue is non-empty on a known sample.
- End-to-end demo: run BinAgent analysis starting from a verified sink; show that evidence and labels guide the agent to relevant code quickly.
- Regression: label IDs remain stable across re-runs (or are mappable via deterministic hashing).

6.12 Stage 12 — End-to-end evaluation and demo protocol (M9)

Goal: produce paper-quality evidence that the approach works, and that obligations materially improve trustworthiness.

6.12.1 Metrics

- Candidate recall: do miners surface true sites (before LLM/verifier filtering)?
- Label precision/recall: measured on VERIFIED labels vs oracle.
- Verified-rate: fraction of LLM proposals that pass obligations (by label type).
- Cost/scale: time per binary, candidates per binary, LLM tokens/cost per binary (budgeted).

6.12.2 Demo correctness checklist

- For each demo claim, show the evidence pack + obligations that justify it.
- Include at least one false-positive that obligations correctly reject (to show fail-closed value).
- Include at least one hard case that becomes PARTIAL/UNKNOWN (to show conservative behavior).

6.12.3 Dataset splits (for generalization claims)

- Architecture split: e.g., ARM Cortex-M vs another MCU family if available.
- Compiler/optimization split: GCC vs Clang (if realistic), O0/O2/O3, with and without LTO.
- Firmware-family split: drivers vs networking stacks vs application logic.

7. Risk register and fallback strategies (updated)

This project is constrained by missing metadata, noisy decompilation, and hardware-specific idioms. The plan explicitly includes fallback paths so progress continues even when a sub-problem is harder than expected.

Risk	Impact	Mitigation	Fallback
CFG/function recovery is too noisy on raw blobs	Downstream miners and slices degrade	Use Ghidra best-effort + confidence tracking; prioritize ELF samples early	Restrict evaluation to T1 (ELF) for main quantitative results; treat T2 as qualitative.
MMIO vs flash address confusion	High FP on MMIO_READ	Memory-map hypothesis scoring + addr-range obligations (O_MMIO_2)	Narrow to MCU families with known maps; report limitations explicitly.
LLM cost/latency dominates	Infeasible to run on many candidates	Top-K ranking + caching + batching; default temperature=0	No-LLM mode: miner+verifier only; run LLM only on a small subset for case studies.
DMA diversity too hard	VS3 slips; weak DMA results	Stage DMA Level 1–2; small curated DMA dataset	Defer Level 3; publish with partial DMA and strong MMIO/ISR results.
LOOP_WRITE too hard	Sink coverage incomplete	Stage last; rely on COPY/MEMSET/STORE first	Report LOOP_WRITE as future work with microbench-only results.
Ground truth labeling is expensive	Weak evaluation	T0 microbench + automated oracle extraction for T1	Focus quantitative claims on T0/T1; do qualitative on T2.

7.1 No-LLM fallback mode (required)

The pipeline must function without LLMs. In no-LLM mode, miners emit heuristic labels with evidence packs; verifier still checks obligations. This mode supports environments with limited inference access and provides a strong baseline for ablations.

- Heuristic labeler: deterministic rules assign MMIO_READ/COPY_SINK when obligations can be satisfied.
- Verifier remains the same; if a heuristic label fails obligations, it is rejected.
- Report results for (heuristic-only + verifier) as a baseline alongside LLM variants.

8. Selected references (for writing the paper)

1. Sun et al., 'LLM-Powered Binary Taint Analyzer (LATTE)', ACM TOSEM, 2025.
2. Jiang et al., 'AdaTaint: A Neuro-Symbolic Framework for Adaptive Taint Analysis', arXiv preprint, 2025.
3. Scharnowski et al., 'Fuzzware: Using Precise MMIO Models for Effective Firmware Fuzzing', USENIX Security, 2022.
4. Feng et al., 'P2IM: Scalable Peripheral Interface Modeling for Firmware Fuzzing', USENIX Security, 2020.
5. Scharnowski et al., 'GDMA: Generic DMA Rehosting for DMA-Dependent Firmware', USENIX Security, 2025.
6. Cui et al., 'DICE: Automatic Emulation of DMA for Firmware Testing', IEEE S&P, 2021.
7. Xia et al., 'TaiE: Improving Function Identification for Monolithic Firmware', ICPC, 2024.
8. Zhao et al., 'LARA: LLM-aided Source Recovery for Taint-based Vulnerability Detection in IoT Web Services', USENIX Security, 2024.

Appendix A — Implementation checklist (quick reference)

- VS0: MMIO_READ end-to-end demo + eval on T0
- VS1: COPY_SINK (API/sig) + MMIO->sink path demo
- VS2: ISR parsing + ISR buffer object label demo
- VS3: DMA Level 1–2 demo (small curated set)
- VS4: MEMSET/STORE sinks incremental
- VS5: LOOP_WRITE microbench only (optional)
- Paper: baseline + ablation tables + cost/scaling + error taxonomy + 2–3 case studies

Appendix B — Case study: the same CWE in unstripped ELF vs stripped ELF vs raw BIN

Purpose: make the "raw .bin vs ELF" discussion concrete by walking through a single vulnerable firmware program, compiled into three artifacts, and showing how the exact same source (hardware input) appears differently before and after disassembly/decompilation. This appendix is meant to be copy-pastable into a paper's motivating example or evaluation section.

B.1 Toy firmware program (CWE-787) and the target source label

We use a minimal Cortex-M0 bare-metal firmware written in C++ (freestanding, no libc). It contains a CWE-787 out-of-bounds write in an interrupt handler. The attacker-controlled byte originates from a UART MMIO data register.

- Target RQ1 object:
  - Source type: MMIO_READ
  - Concrete site: read from address 0x40013804 (UART data register)
  - Context: inside ISR (vector table entry -> handler)

Relevant code excerpt (file: cortexm_demo_mmio_source.cpp):

```
static inline uint8_t UART_ReadByte(void) {

    volatile uint32_t* const UART_DR = (volatile uint32_t*)0x40013804u;

    return (uint8_t)(*UART_DR & 0xFFu);

}



volatile uint8_t g_rx_buf[16];

volatile uint32_t g_rx_len = 0;



extern "C" void USART1_IRQHandler(void) {

    uint8_t b = UART_ReadByte();  // <-- SOURCE: MMIO_READ

    g_rx_buf[g_rx_len++] = b;     // <-- CWE-787 OOB write when g_rx_len >= 16

}
```

B.2 Build artifacts (three formats) and why we need all three

From the same source file, we build: (i) an unstripped ELF, (ii) a stripped ELF, and (iii) a raw stripped .bin. The goal is not to run the firmware, but to analyze how a static recovery pipeline sees the same source under different packaging.

Representative build commands:

```
clang++ --target=arm-none-eabi -mcpu=cortex-m0 -mthumb \
  -ffreestanding -nostdlib -fno-exceptions -fno-rtti \
  -fno-unwind-tables -fno-asynchronous-unwind-tables -fuse-ld=lld \
  -Wl,-T,cortexm.ld -Wl,--gc-sections -O2 -g0 \
  cortexm_demo_mmio_source.cpp -o cortexm_demo_unstripped.elf


llvm-objcopy --strip-all cortexm_demo_unstripped.elf cortexm_demo_stripped.elf
llvm-objcopy -O binary --only-section=.isr_vector --only-section=.text --only-section=.ARM.exidx \
  cortexm_demo_stripped.elf cortexm_demo_stripped.bin
```

B.3 What changes before decompilation (metadata and bootstrapping)

Before any decompilation, the three formats differ in what "ground truth" metadata they expose to analysis tools. This directly affects how much bootstrap work we must do to even obtain a correct CFG and call graph.

Table B.1 summarizes the key differences relevant to recovering an MMIO_READ source.

Aspect	Unstripped ELF	Stripped ELF	Raw BIN (.bin)
Arch/endian	Explicit in ELF header (ARM, little-endian). Tools auto-configure.	Same as unstripped (ELF header still present).	Absent. Must infer (e.g., vector table/SP range, ISA heuristics, strings).
Load address / entry	Explicit (program headers, entry point).	Explicit (same as unstripped).	Absent. Must infer base address (e.g., reset vector) and map flash/RAM.
Sections (.text/.data)	Explicit boundaries. Disassembler can separate code vs data more reliably.	Explicit boundaries (even after stripping).	Absent. Must infer code/data boundaries; literal pools are easy to mis-disassemble.
Symbol names (ISR/function/global)	Present. ISR = USART1_IRQHandler, globals =	Removed. Functions become FUN_08000090-style;	Removed + no symbol table at all.

	g_rx_buf/g_rx_len.	globals become DAT_2000xxxx-style.	
Vector table	Present as .isr_vector section; trivial to locate.	Present as bytes in .isr_vector section; trivial to locate.	Present only as “first bytes” (if firmware is Cortex-M-like). Need heuristics for table length/validity.
False-positive pressure for “MMIO constants”	Lower: sections + better xrefs help exclude random data.	Medium: still have sections, but names gone; analysis must rely on def-use evidence.	High without bootstrap: naive 0x4000xxxx scans are extremely noisy; must tie constants to load/store semantics.

B.4 What the “same source” looks like after disassembly and decompilation

In this example, the source is the MMIO read of UART_DR at address 0x40013804 in the ISR body. The underlying machine code is identical across formats; what changes is how easily tools can recover structure and names.

Unstripped ELF (symbolized disassembly excerpt, representative):

```
08000090 <USART1_IRQHandler>:

  8000090: ldr r0, [pc, #0x10]   ; loads literal 0x40013804

  8000092: ldr r0, [r0]         ; <-- MMIO READ (source candidate)

  ...

  80000a4: .word 0x40013804      ; UART_DR
```

Stripped ELF: the same bytes exist, but symbol names are gone. A robust disassembler/decompiler (e.g., Ghidra) can still mark the literal pool word as data due to the PC-relative xref, but naive linear disassembly may decode it as instructions. Our pipeline avoids this by using IR-level memory semantics (P-code) to identify that the loaded constant is used as a pointer in a subsequent LOAD.

Raw BIN: the same bytes exist, but there is no header to tell the tool where flash is mapped. Once the bootstrapper infers base address 0x08000000 from the reset vector and creates flash/RAM blocks, the downstream representation converges to the stripped-ELF case. Without bootstrap, even the vector table and constant pools may be misinterpreted.

Representative decompiler-level pseudocode (what we want to recover):

```
// Unstripped: names preserved
```

```
void USART1_IRQHandler() {
    uint8_t b = *(volatile uint32_t*)0x40013804u;
    g_rx_buf[g_rx_len++] = b;  // CWE-787 (OOB write)
}


// Stripped/raw: names lost, but semantics still recoverable
void FUN_08000090() {
    uint32_t v = *(uint32_t*)0x40013804u;   // still visible as constant + LOAD
    DAT_20000000[DAT_20000010++] = (uint8_t)v;
}
```

B.5 When problems appear in stripped/raw binaries: tooling vs custom methods vs LLM

Table B.2 lists the specific problems that arise when moving from unstripped ELF to stripped ELF and then to raw .bin, and clarifies what is solved by off-the-shelf tools, what requires custom pre-processing/analysis, and where LLM is helpful (and where it is not).

Problem (impact on source recovery)	Solved by existing tools?	Custom method needed in RQ1?	Can LLM help? (role)
Missing symbol names (stripped ELF / raw bin): harder to name ISR and infer intent.	Yes (decompilers still work), but semantics/names are lost.	Yes: we require evidence-grounded labeling and verified summaries.	Yes: LLM can propose semantic labels from evidence; verifier prevents hallucination.
No arch/endian/entry metadata (raw bin): tool cannot even disassemble correctly.	Partially: tools can load raw binaries but typically require manual configuration.	Yes: raw-binary bootstrapper (vector-table and heuristic inference).	Limited: LLM can assist MCU-family guessing from strings/constants, but cannot replace hard constraints.
No section boundaries (raw bin): literal pools and jump tables mis-disassemble -> false	Partially: advanced tools can recover with heuristics, but quality depends on correct base/map.	Yes: memory-map hypothesis + xref-driven data marking; IR semantics not linear sweep.	Some: LLM may classify a constant as likely MMIO vs ROM table, but must be verified.

sources.			
MMIO constant appears as MOVW/MOVT rather than a literal pool (common on Thumb-2): source miner must handle both.	Yes in good IR (decompiler sees the constant), but not in naive literal scans.	Yes: extend miner patterns beyond “literal pool only”.	Minor: LLM not needed; this is deterministic instruction semantics.
DMA-backed input (raw bin): DMA writes to RAM mean there is no local LOAD to tag as source.	No (static, general): most DMA work is dynamic/rehosting-driven (e.g., DICE/GDMA).	Yes: staged DMA detection (config-site -> dst-pointer slice -> consumer evidence) + obligations.	Helpful-but-insufficient: LLM can interpret config patterns and infer direction (RX vs TX), but must pass verifier obligations.

Takeaway: for the MMIO_READ family, stripped raw .bin is feasible with a deterministic bootstrap (vector table + base/map inference). For DMA-backed sources, the absence of an explicit LOAD instruction makes the problem fundamentally harder, especially without types; LLM can help interpret patterns, but a verifier is essential to keep precision.